

May 6, Station F,
Paris

Skill Foundry: An AI That Embodies a Robot, Watches Itself, and Learns

Charbel DANDJINO
AI Research Engineer
CTO - Co-founder, Steath



You mean Physical AI?

- Object Detection - tracking -segmentation
- event detection
- scene analysis
- etc



Physical AI

Embodied AI

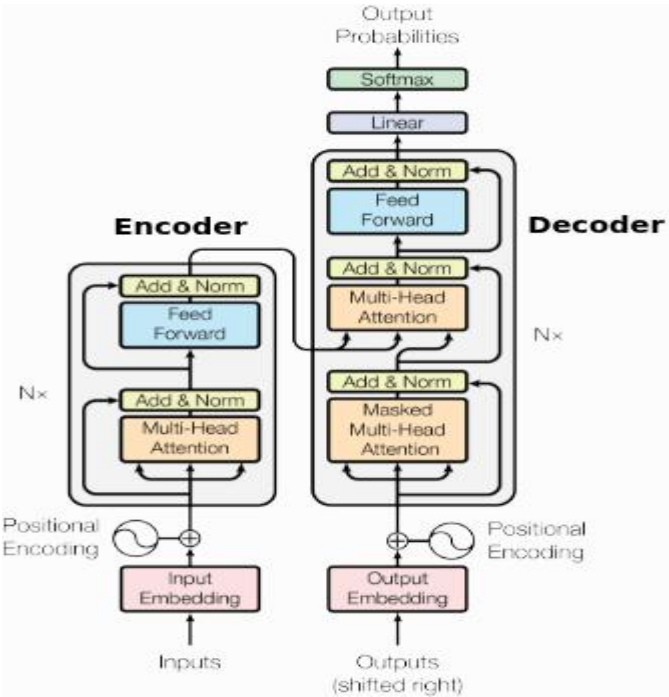


- AI is aware of being situated in a physical body
- can take actions in the physical world.



what changed in AI for robotics, and why is it becoming so interesting now?

Transformer



- Scale very well with data and compute.
- revolution in language model then vision and almost any modality ,

so it has be tried for robotic

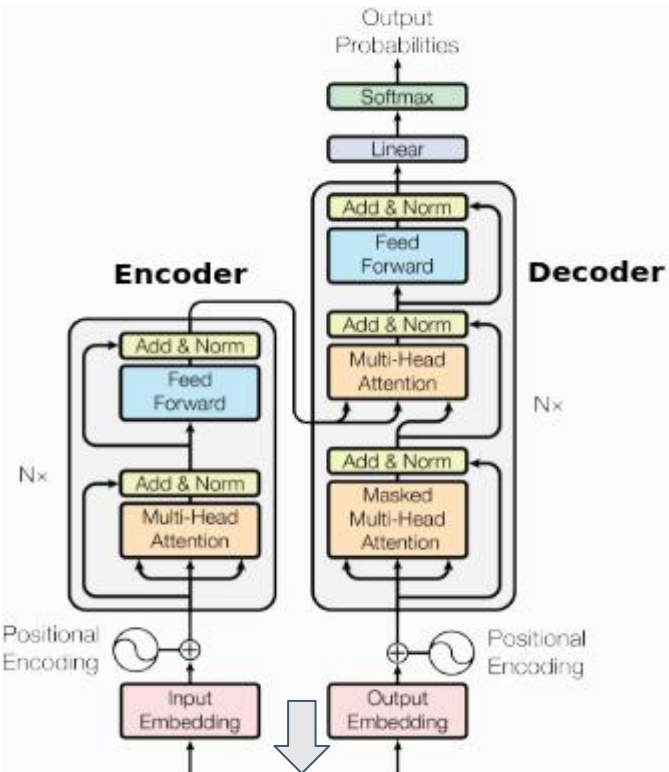


Generalist transformer

Embodied MLM

Robot transformer

Transformer

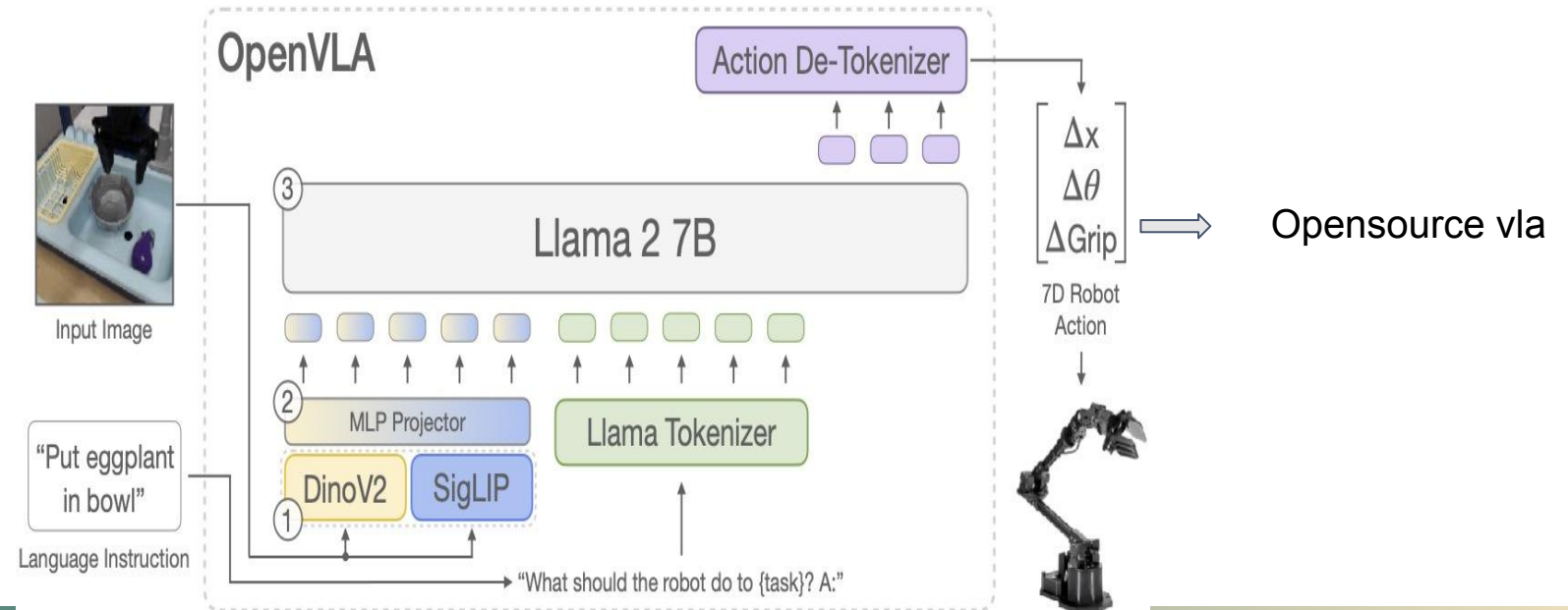


so it has be tried
for robotic

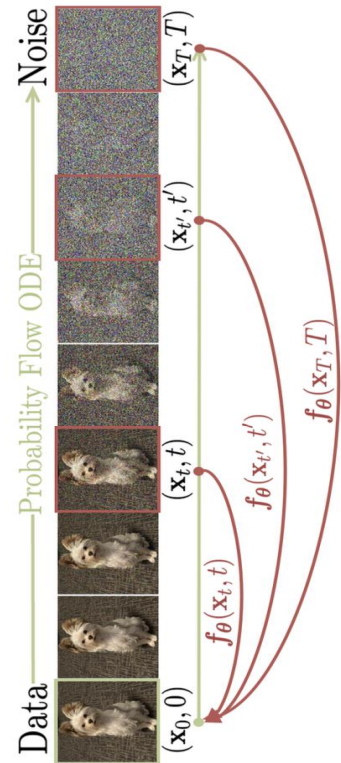
- Scale very well with data and compute.
- revolution in language model then vision and almost any modality ,



→ First VLA

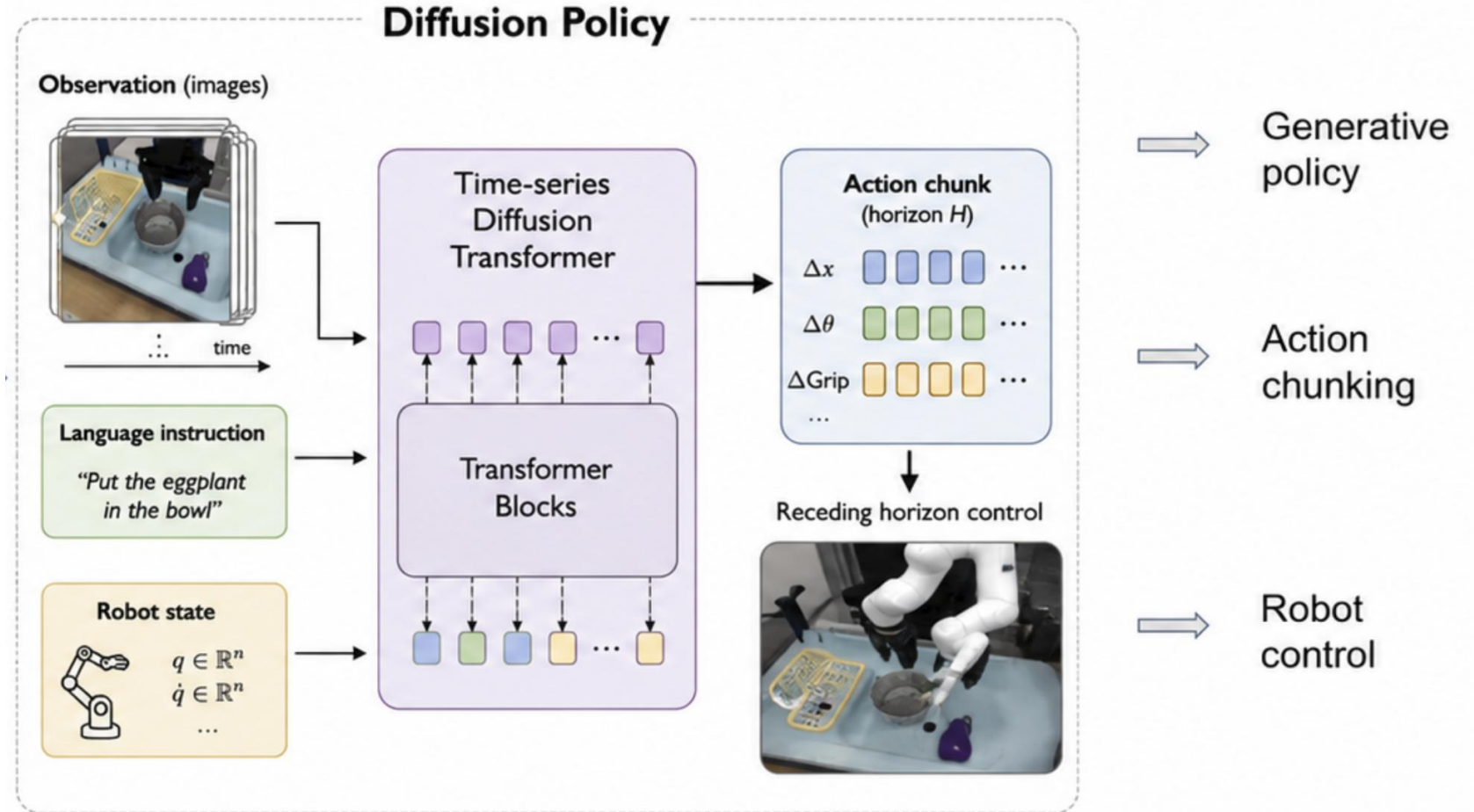


Diffusion

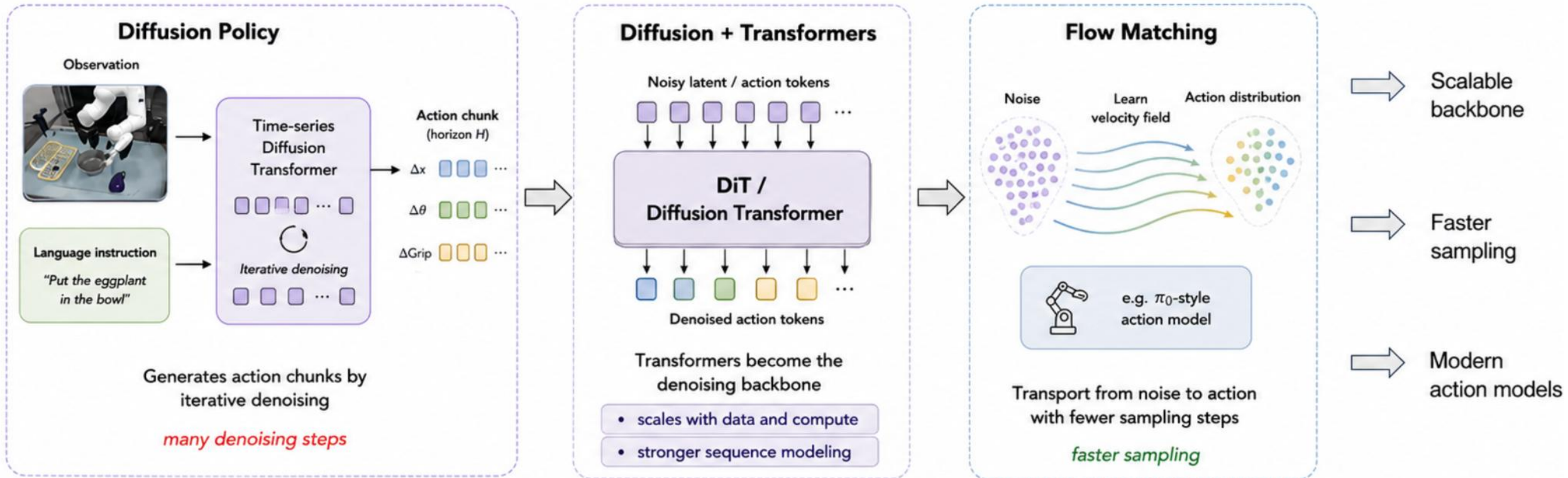


also used for
robotics

- Samples by iterative denoising
- Very successful in generative AI
- In robotics: used for action or trajectory generation

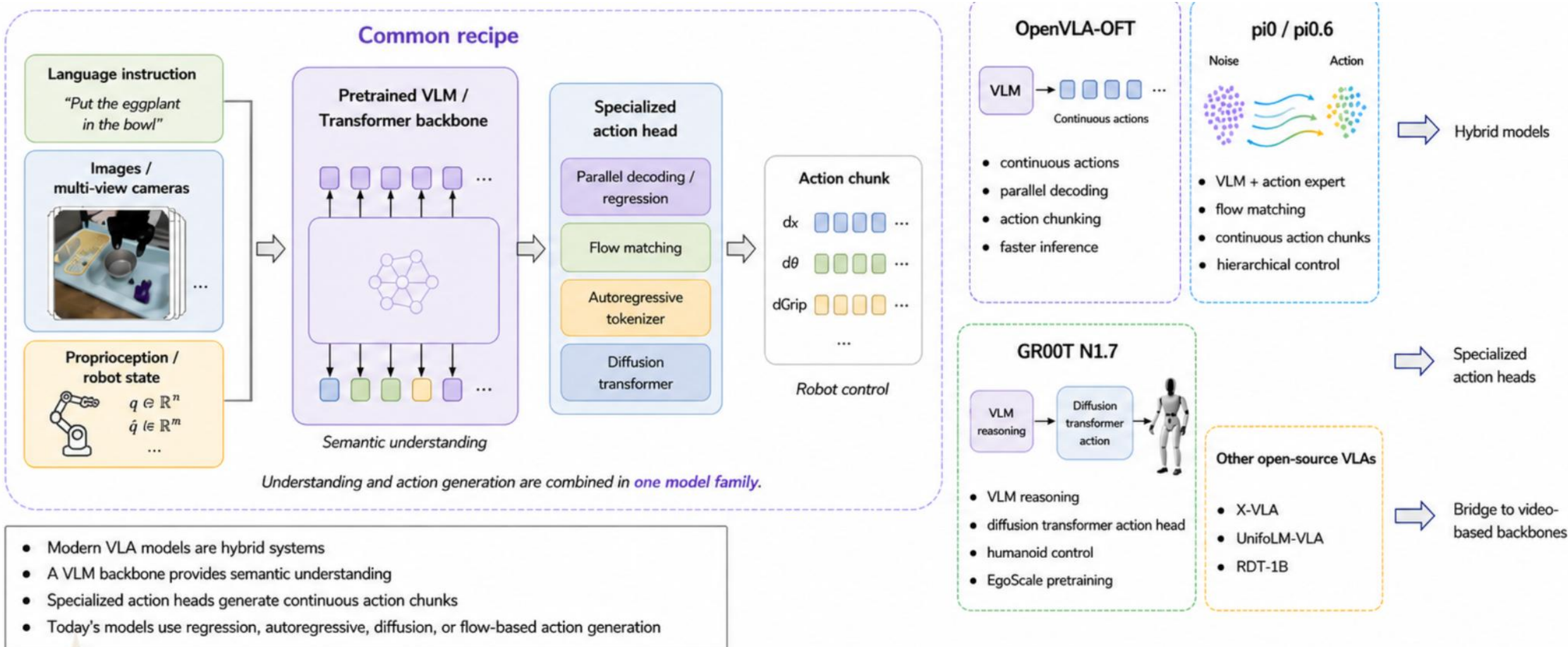


From Diffusion to Flow Matching



- Diffusion policies are powerful, but often slow at inference
- Transformers became the denoising backbone (DiT-style models)
- Flow matching learns a velocity field from noise to action
- Modern robot policies now use diffusion / DiT / flow matching





~~Transformers scaled in language and vision~~, so robotics tried the same direction.

Transformer architectures scale well with compute and data. Robotics borrowed that recipe and turned perception + language into action.



This gives us more capable robot policies - but not yet an embodied experiment operator.



Diffusion policies

- Sample action trajectories
- Handle multimodal behaviors
- Useful for manipulation and control

Video / DiT-style backbones

- Borrow priors from generation
- Learn motion and physical regularities
- Can accelerate convergence

World models

- Predict future states
- Can be action-conditioned
- Support planning and evaluation

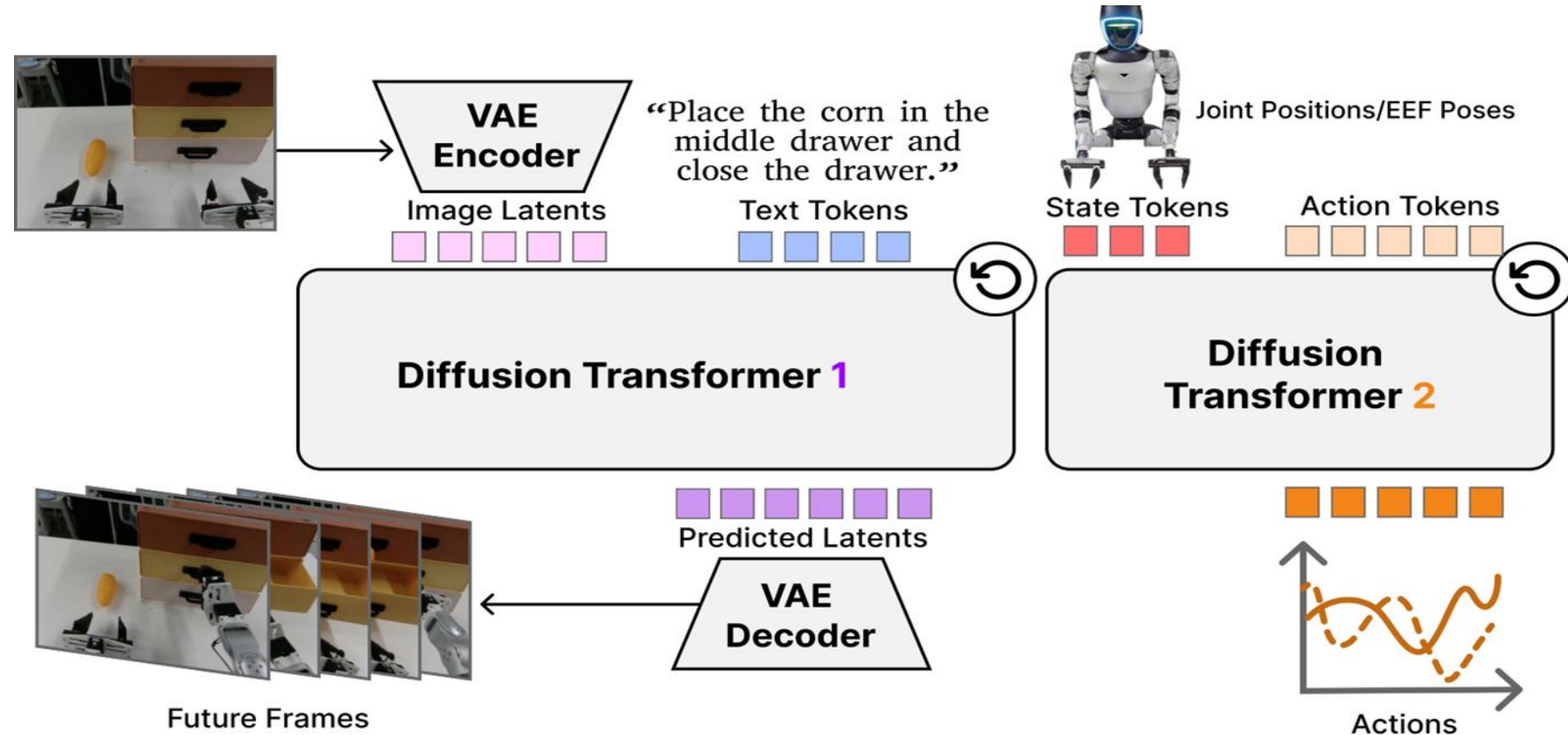
The modern robot stack is becoming a toolbox of models - not a single monolith.



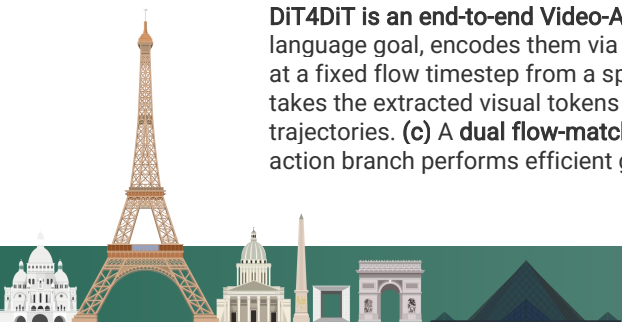
Diffusion, video backbones, and world models changed the design space.

Video / DiT-style backbones

Borrow priors from generation
Learn motion and physical regularities
Can accelerate convergence



DiT4DiT is an end-to-end Video-Action Model with a cascaded dual-DiT architecture. (a) The **Video DiT** (initialized from Cosmos-Predict2.5-2B) takes the current observation frames and a language goal, encodes them via a causal video VAE into latent space, and models future visual dynamics via flow matching. A forward hook mechanism intercepts intermediate hidden activations at a fixed flow timestep from a specific transformer layer, converting the generative process into rich, physics-aware visual tokens – without requiring full video reconstruction. (b) The **Action DiT** takes the extracted visual tokens via cross-attention, along with proprioceptive state embeddings and noisy action trajectories, and predicts a velocity field to generate precise robot action trajectories. (c) A **dual flow-matching objective** with a tri-timestep scheme jointly optimizes both branches end-to-end, allowing the video branch to learn the full generative trajectory while the action branch performs efficient generative inverse dynamics.



Beyond VLA: generative action models

GOSIM Paris 2026

Diffusion, video backbones, and world models changed the design space.

Video / DiT-style backbones

Borrow priors from generation
Learn motion and physical regularities
Can accelerate convergence

Diverse, Non-repetitive Pretraining Data



Task Diversity > Repetition

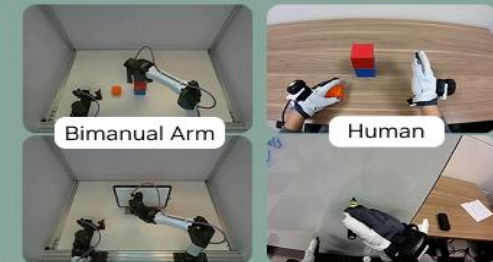


Generalization after Post-training

DreamZero

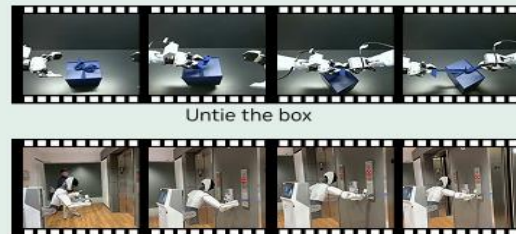
Videos Language Proprio

World Action Model

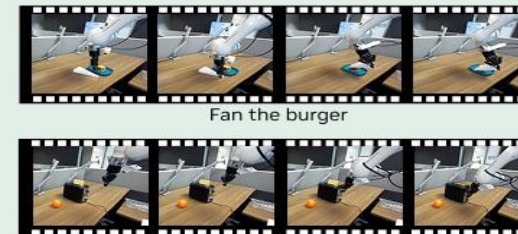


Task Transfer via X-Embodiments

Zero-shot Generalization to Unseen Tasks in Unseen Environments



Press elevator button



Depress the lever on the toaster

Few-shot Adaptation to New Embodiment



With only 30 minutes of play data



LLMs revived RL culture, and robotics benefits from it.

RLHF

align behavior from feedback

GRPO / RLVR

stronger reward-driven
optimization

Sandbox agents

learn inside controlled
environments

Agent memory

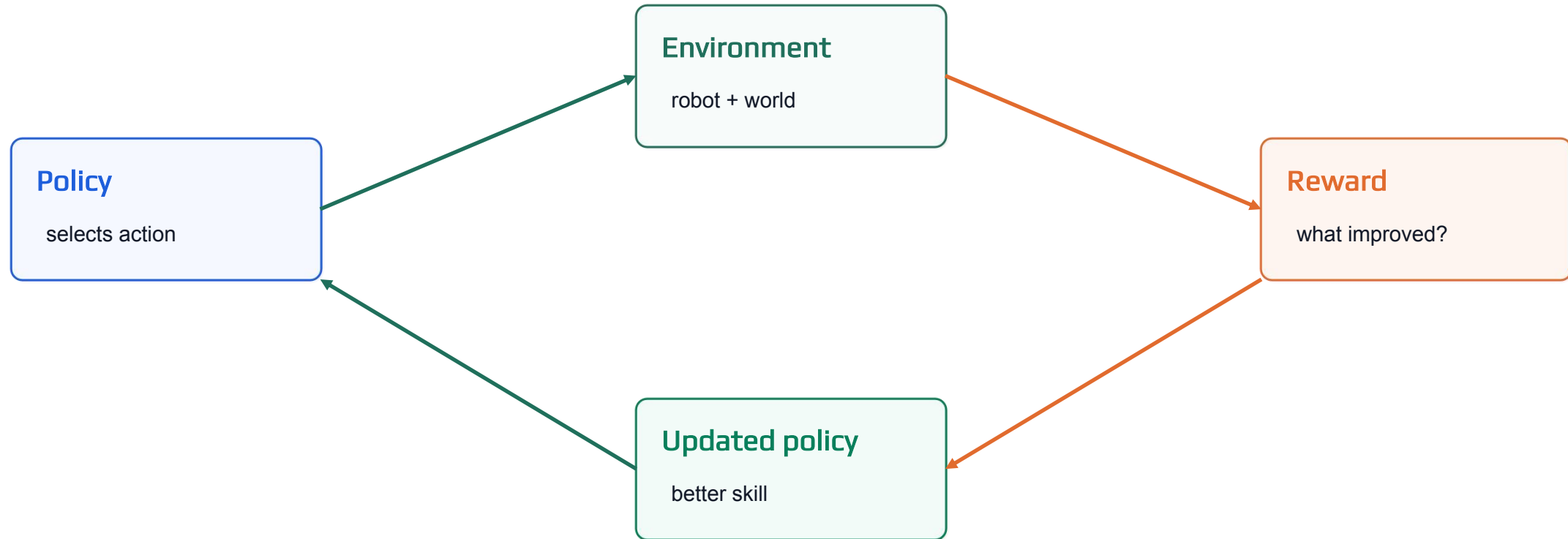
retain lessons across attempts

In robotics, RL is not a trend callback. It is still one of the few engines that can discover motor strategies we did not explicitly program.



Why RL still matters for robots

RL turns trial, reward, and embodiment into motor skills.

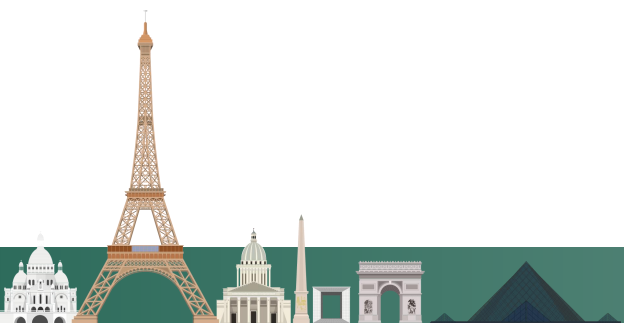
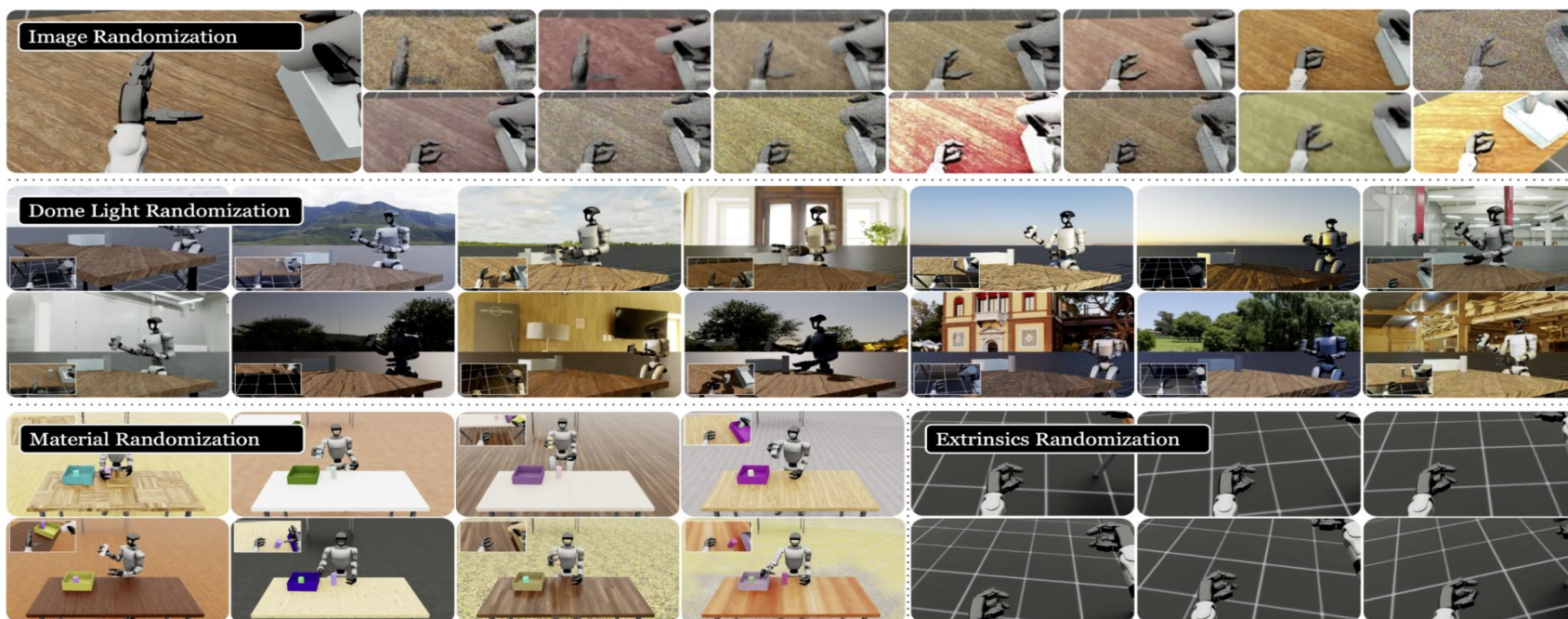
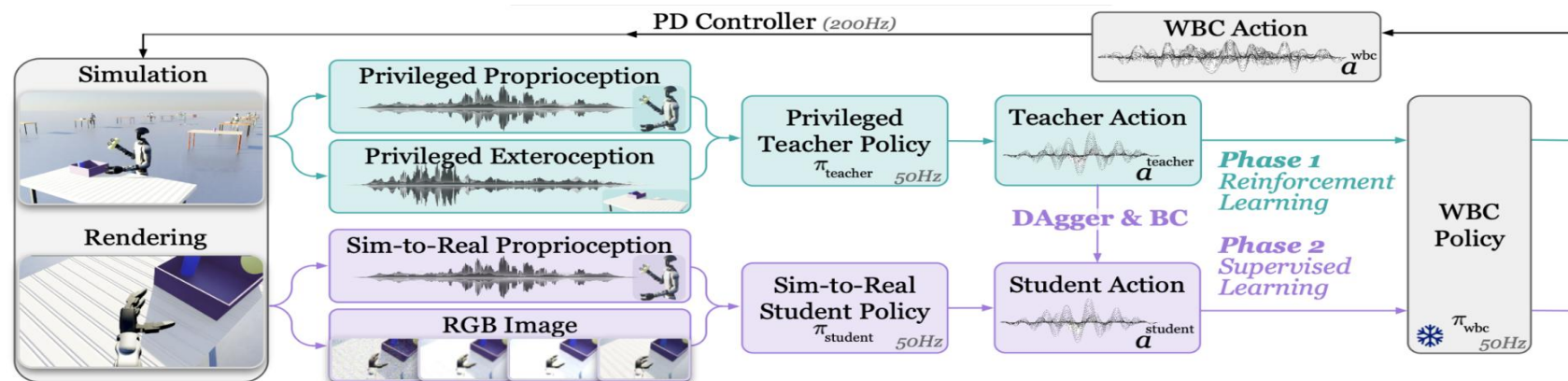


Humanoid locomotion • whole-body control • dynamic recovery • obstacle avoidance • manipulation primitives

But RL depends on the training world and on the reward.



- can solve data bottleneck for non deformable objects
- with method like privileged teacher distillation (dagger) and sim to real progress and compute we can learn strong visuo motor policy (Doorman)
- sim also getting better for deformable object
- opensource differentiable simulation
- RL for vla postraining becoming common pattern like for LLM





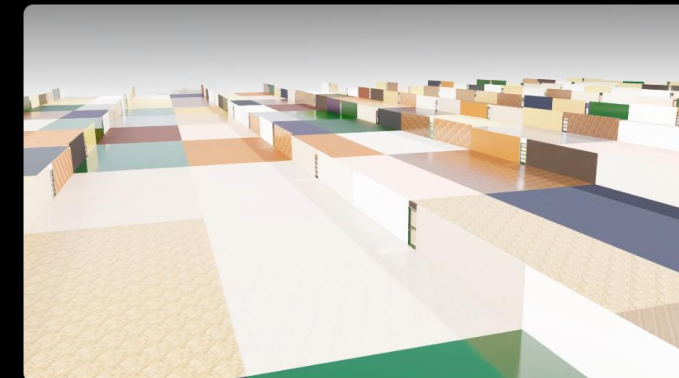
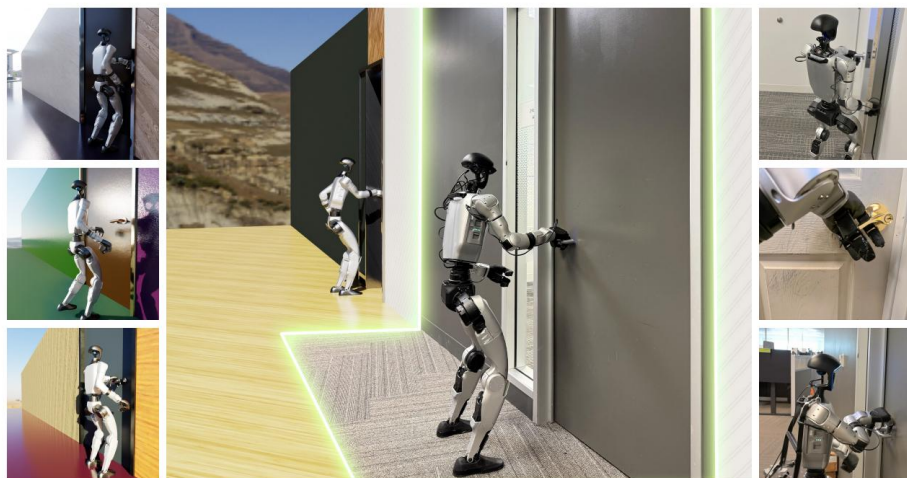
2025-12-2

Opening the Sim-to-Real Door for Humanoid Pixel-to-Action Policy Transfer

Haoru Xue^{1,2*}, Tairan He^{1,3*}, Zi Wang^{1*}, Qingwei Ben^{1,4}, Wenli Xiao^{1,3}, Zhengyi Luo¹, Xingye Da¹, Fernando Castañeda¹, Guanya Shi³, Shankar Sastry², Linxi "Jim" Fan^{1†}, Yuke Zhu^{1†}

¹NVIDIA ²UC Berkeley ³CMU ⁴CUHK

*Equal Contribution [†]Project Leads



Teacher-Student-Bootstrap

We apply GRPO fine-tuning to bootstrap the student on top of classical teacher-student distillation. We find this stage to be uniquely useful for challenging loco-manipulation tasks due to partial observability. The reward is mostly binary on the task success criteria. The result: 20-30% success rate improvement.



Phase 1: Train privileged teacher with PPO
(1 L40s GPU, 6 hours)



Distill



Phase 2: Distill onto vision student with DAgger
(32 L40s GPUs, 24 hours)



Copy initialize



Phase 3: Fine-tune student with GRPO
(64 L40s GPUs, 12 hours)



Perceptive Humanoid Parkour: Chaining Dynamic Human Skills via Motion Matching

Zhen Wu^{*1}, Xiaoyu Huang^{*1,2}, Lujie Yang^{*1}, Yuanhang Zhang^{1,3}, Koushil Sreenath^{1,2}, Xi Chen¹, Pieter Abbeel^{†1,2}, Rocky Duan^{†1}, Angjoo Kanazawa^{†1,2}, Carmelo Sferrazza^{†1}, Guanya Shi^{†1,3}, C. Karen Liu^{†1,4}

¹Amazon FAR, ²UC Berkeley, ³CMU, ⁴Stanford University, [†]Amazon FAR team co-lead

Page: <https://php-parkour.github.io/>

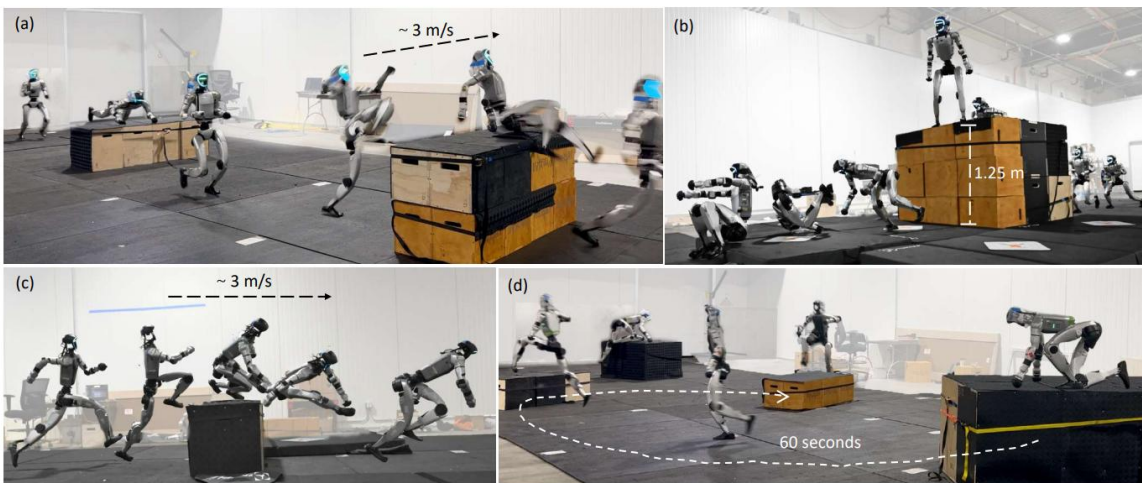


Fig. 1: **Perceptive Humanoid Parkour (PHP)** enables a Unitree G1 humanoid robot to execute highly dynamic, long-horizon parkour behaviors using onboard perception. By composing various agile human skills via motion matching and a teacher-student training pipeline, we train a single multi-skill visuomotor policy capable of complex contact-rich maneuvers including (a) cat-vaulting over a short obstacle followed by dash-vaulting over a higher obstacle at approximately 3 m/s, (b) climbing onto a 1.25 m (96% of robot height) wall, and rolling down, (c) speed-vaulting over an obstacle at approximately 3 m/s, and (d) a 60-second continuous traversal of a complex parkour course with autonomous skill selection and seamless transitions.

OmniRetarget: Interaction-Preserving Data Generation for Humanoid Whole-Body Loco-Manipulation and Scene Interaction

Lujie Yang^{*1,2}, Xiaoyu Huang^{*1,3}, Zhen Wu^{*1}, Angjoo Kanazawa^{†1,3}, Pieter Abbeel^{†1,3}, Carmelo Sferrazza^{†1}, C. Karen Liu^{†1,4}, Rocky Duan^{†1}, Guanya Shi^{†1,5}
¹Amazon FAR (Frontier AI & Robotics), ²MIT, ³UC Berkeley, ⁴Stanford University, ⁵CMU

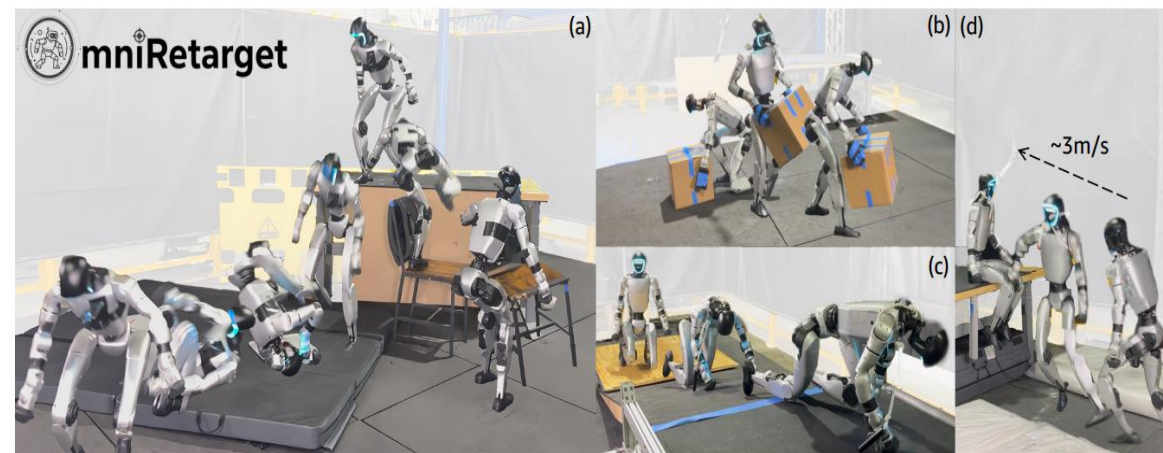


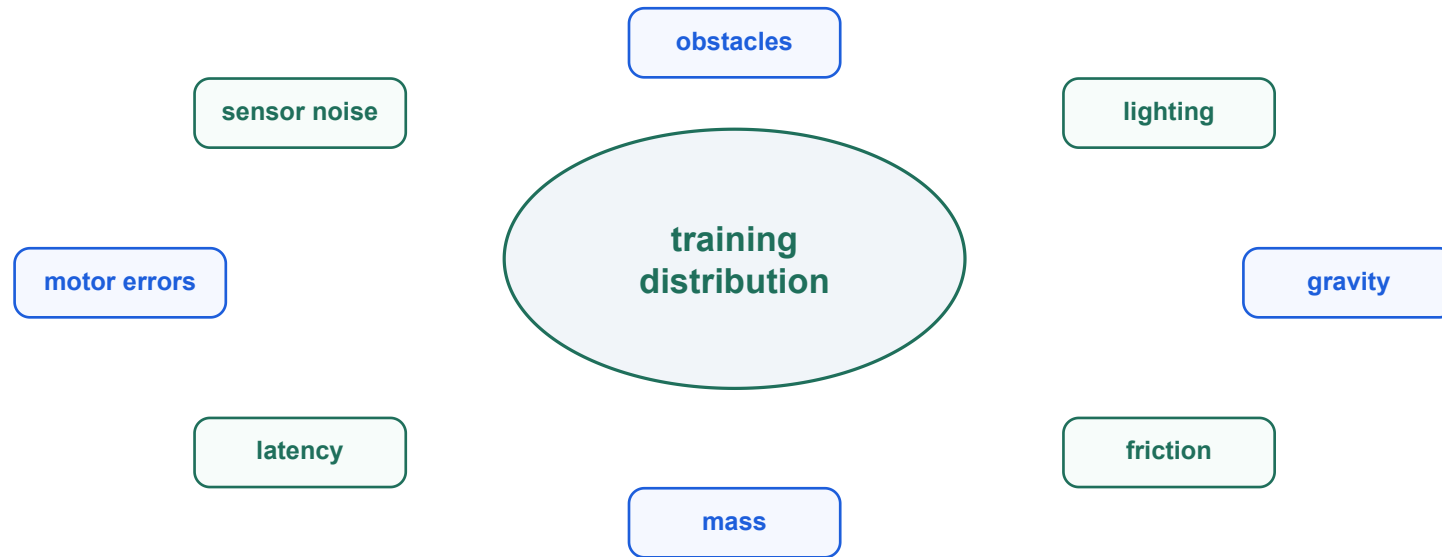
Fig. 1: **OMNIRETARGET** enables reinforcement learning policies to learn complex, long-horizon loco-manipulation skills in challenging environments that transfer zero-shot from simulation to a Unitree G1 humanoid. Thanks to the high-quality interaction-preserving motion retargeting, these policies are trained and deployed in a *minimal and unified* way: it involves only 5 rewards, 4 robot domain randomization terms, and a purely proprioceptive observation space, shared by all tasks. Demonstrated behaviors include (a) 30-second parkour course involving chair moving, stepping & vault, and jump & roll, (b) object transportation, (c) crawling on a slope, and (d) fast platform climbing and sitting.



Simulation became the body lab

Domain randomization turns sim-to-real into an engineering distribution problem.

Instead of one perfect simulator, train across many slightly different worlds.

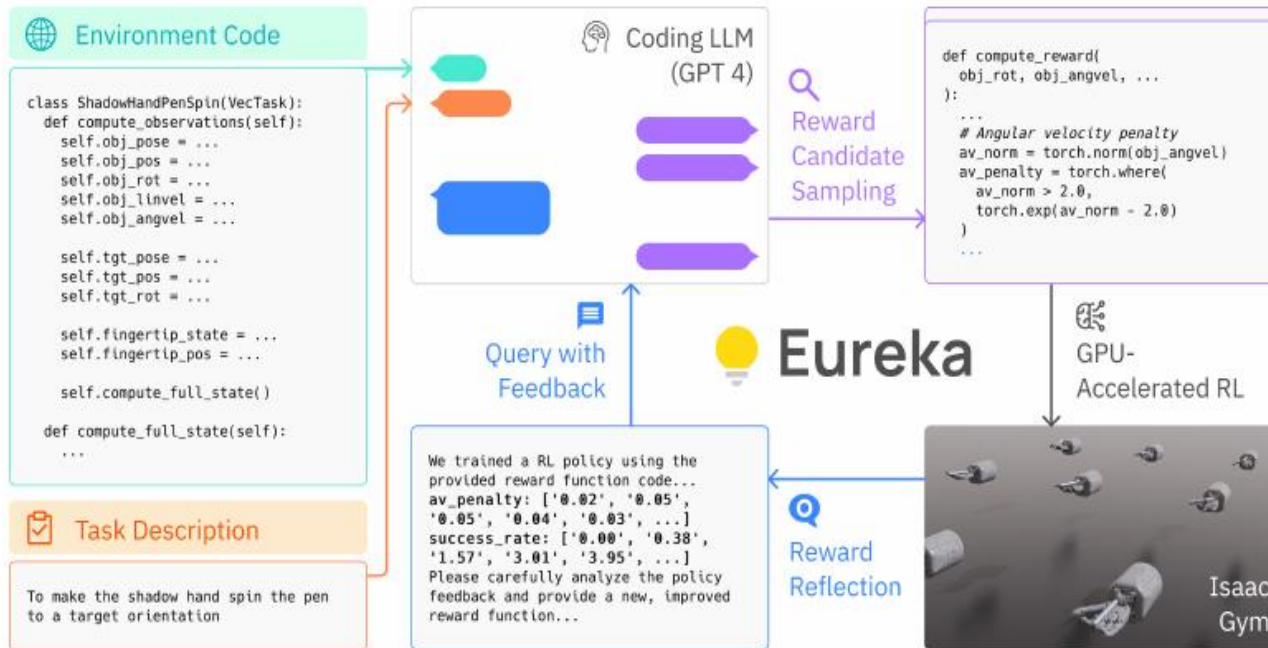


Reality becomes one sample inside the distribution the agent already survived.



Eureka was a key precedent

An LLM can improve rewards from reinforcement-learning feedback.



Eureka loop

LLM writes reward
RL trains policy
Scalar metrics come back
LLM edits reward

Key precedent: language models can search over behavioral incentives faster than manual editing.

But in robotics, the agent also needs to know what the robot actually did.



Metrics tell you whether something changed. Rollouts tell you why.

Metrics-only feedback

success_rate = 0.42
reward improved
collision count changed
loss curves look stable



Embodied rollout feedback

the shoulder clipped the obstacle
the turn was too late
the motion was unstable
the grasp slipped after contact

The feedback channel must include metrics + robot POV + scene view + proprioception + rollout analysis.

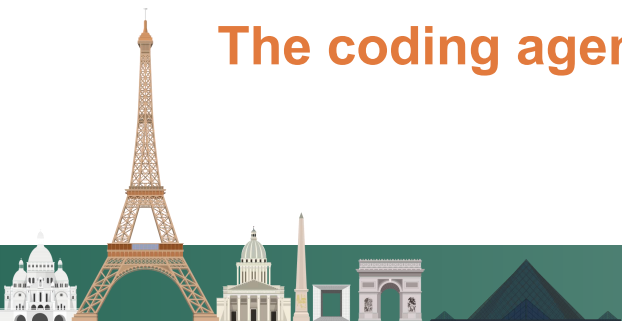


The shift is not only automation. It is continuity across code, training, vision, and memory.

**The same model can write the code,
run the experiment,
watch the robot,
and decide the next iteration.**

- Read the repo and task implementation
- Modify rewards, curriculum, and parameters
- Launch and monitor training
- Deploy the policy as a skill
- Inspect video and proprioception
- Store lessons in memory

The coding agent becomes the missing glue between robotics software, simulation, training, and evaluation.



This is the part that makes Skill Foundry embodied.

**The robot is your body.
Simulation is your training ground.
Your mission is to evolve before
entering the physical robot.**

When you hit a limitation, you can code.

When a behavior fails, you can train.

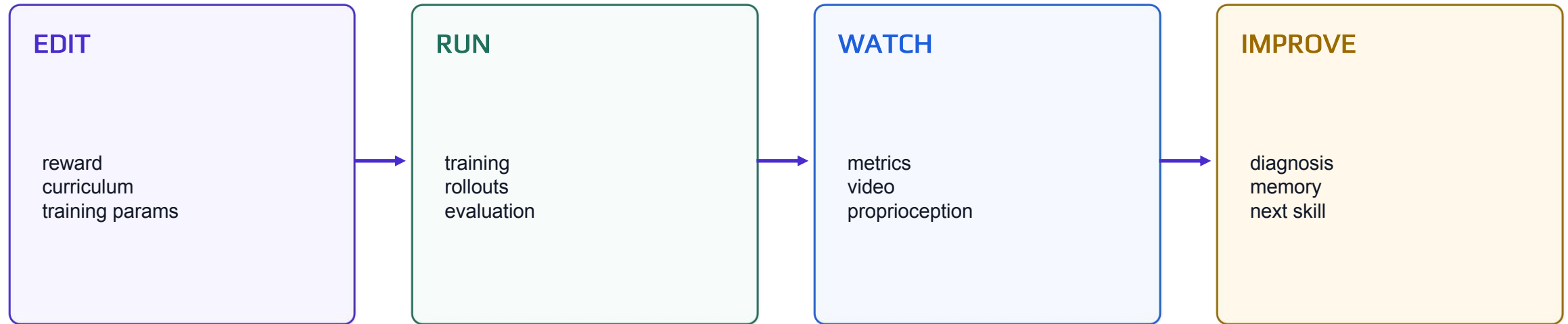
When a skill is missing, you can create it.

Each new skill should add capability, not duplicate existing behavior.



Coding agents became experiment operators

The shift is continuity across code, training, vision, and memory.



The same model can write the code, operate the experiment, inspect the rollout, and choose the next intervention.



If the model can train policies and act as a planner, why not give it evolving objectives?

Give the agent tasks it cannot solve yet.

When a skill is missing, it trains a new one.
When a skill overlaps, it searches for something orthogonal.
When skills compose, it can solve more complex tasks.

Mission

evolve before entering the real robot

Goal

collect useful, orthogonal skills

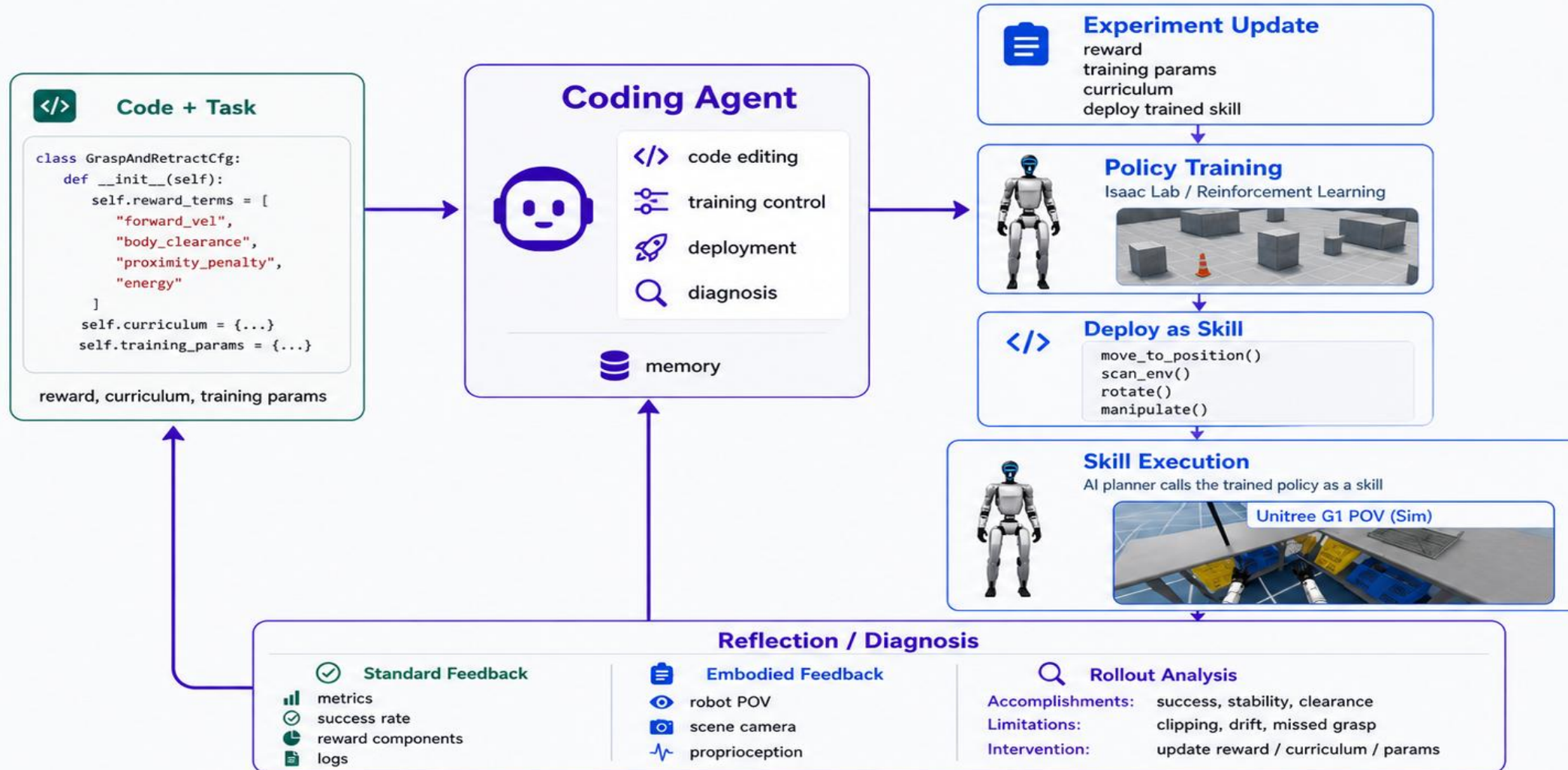
Mechanism

train, execute, observe, remember



Skill Foundry

Keeps the standard Eureka feedback loop and extends it with embodied feedback and rollout analysis



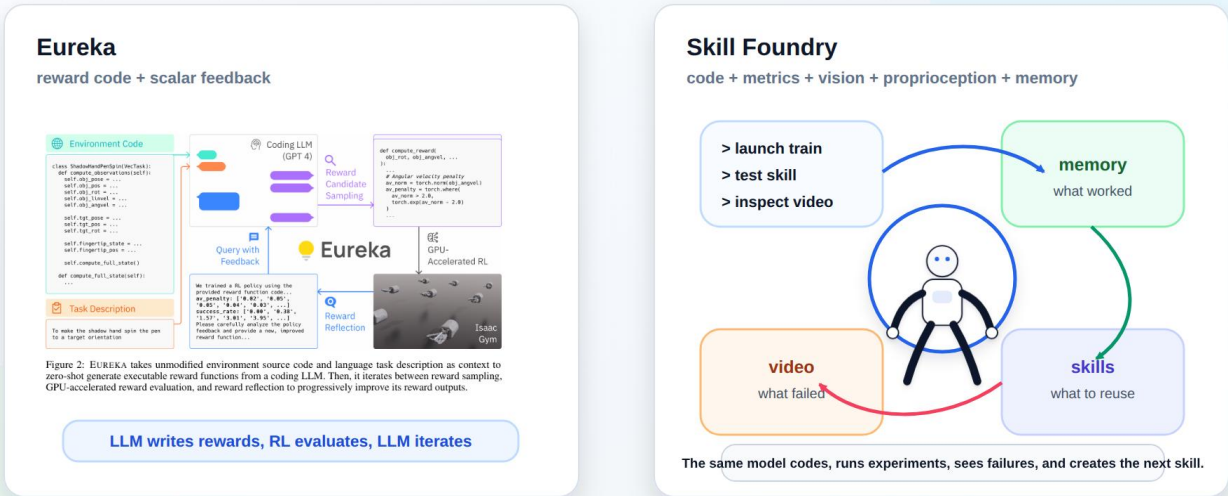
Eureka is the precedent. Skill Foundry extends the loop to embodiment.

Eureka optimizes rewards.

Skill Foundry optimizes embodied skill acquisition.

Eureka vs Skill Foundry

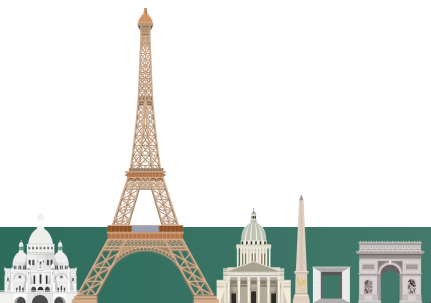
Eureka optimized rewards. Skill Foundry optimizes the experiment loop.



Eureka: reward code + scalar metrics.

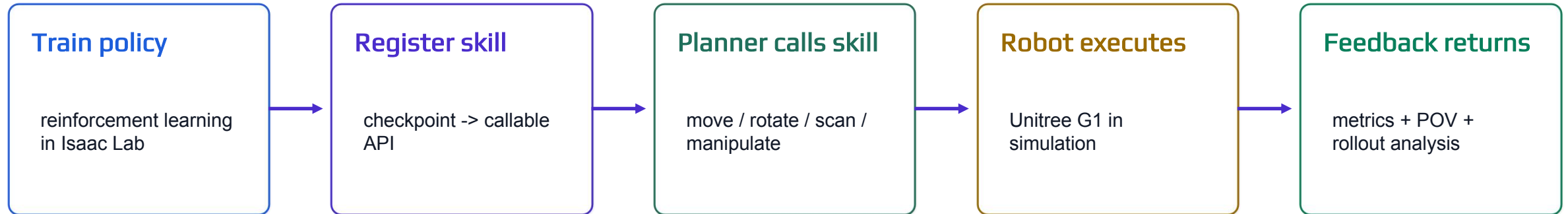
Skill Foundry: code + metrics + video + proprioception + memory + skill library.

The target is not just a better reward. It is a better robot skill.



A trained policy becomes a skill

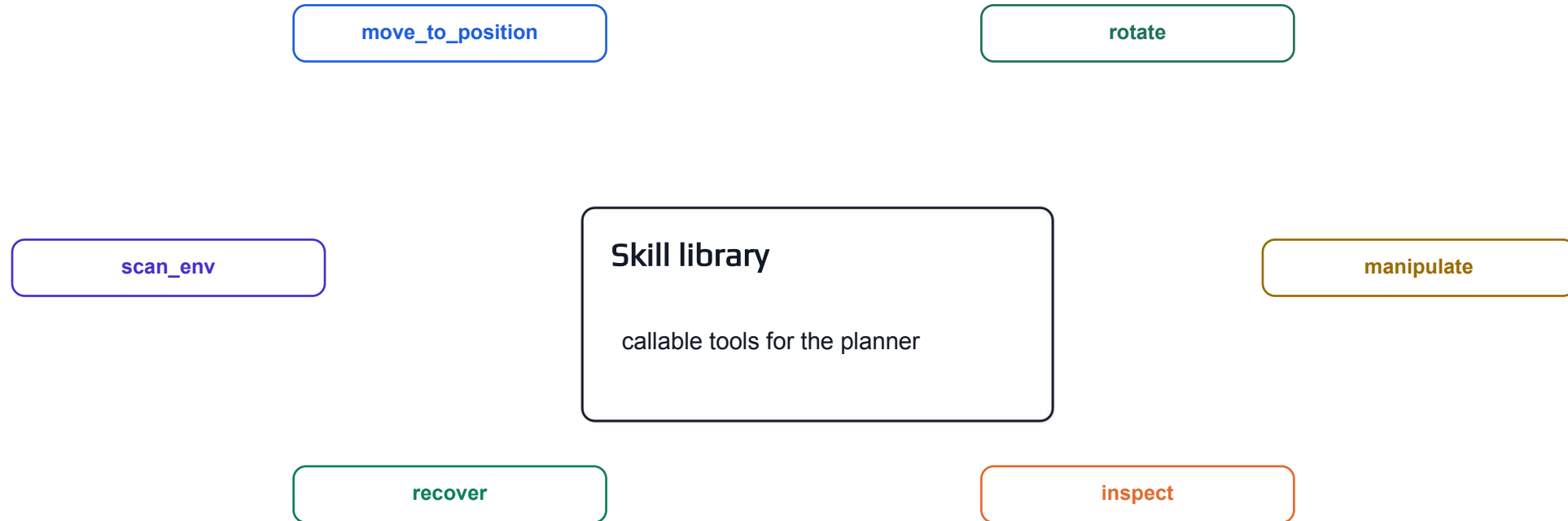
The planner does not become the motor controller. It calls trained skills.



The AI planner drives the robot through a trained policy skill, then learns from what the rollout reveals.



Each new skill should add capability, not duplicate behavior.



The inventory evolves toward coverage: navigation, perception, manipulation, recovery, inspection.



Memory turns runs into evolution

GOSIM Paris 2026

The agent should not rediscover the same lesson every run.

Working memory

current run
active hypothesis

Long-term memory

rewards tried
results and fixes

Spatial memory

where failures
happen in scenes

Skill memory

what exists
what overlaps
what is missing

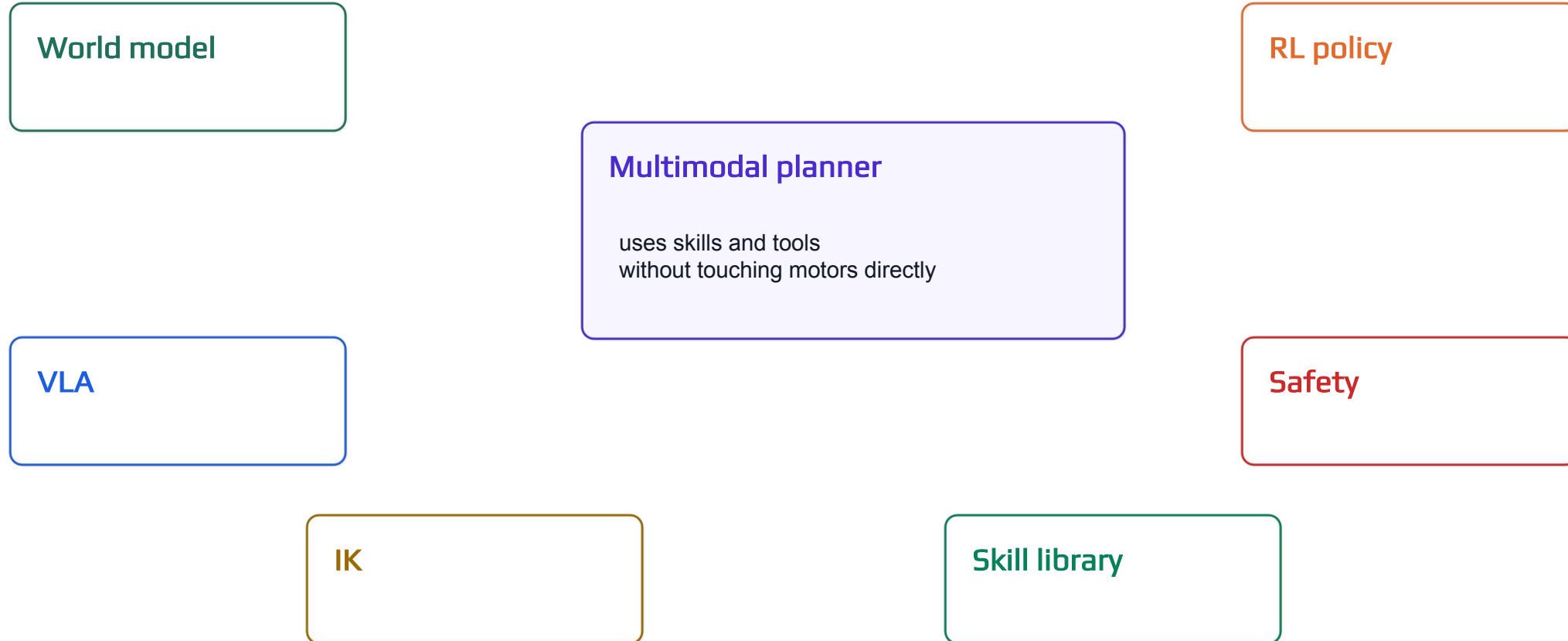
Without memory, every run is isolated. With memory, experiments accumulate into skills.



The larger bet: hierarchical embodied AI

GOSIM Paris 2026

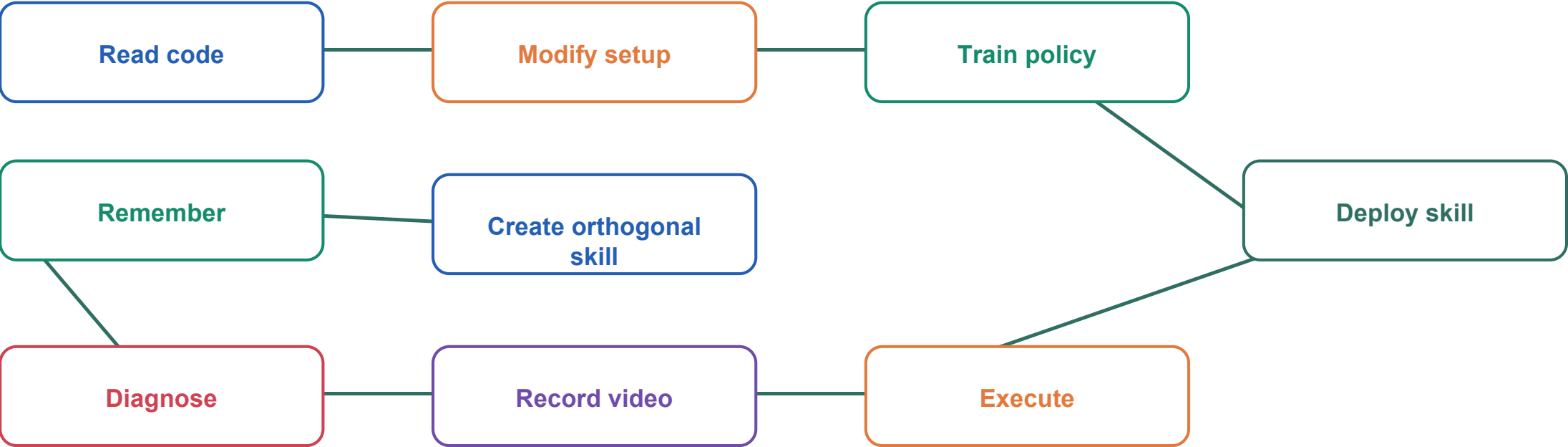
We do not believe one end-to-end model should be the whole robot brain.



Skill Foundry is the evolution layer that grows the skill library.



A full embodied learning loop, not just a reward prompt.

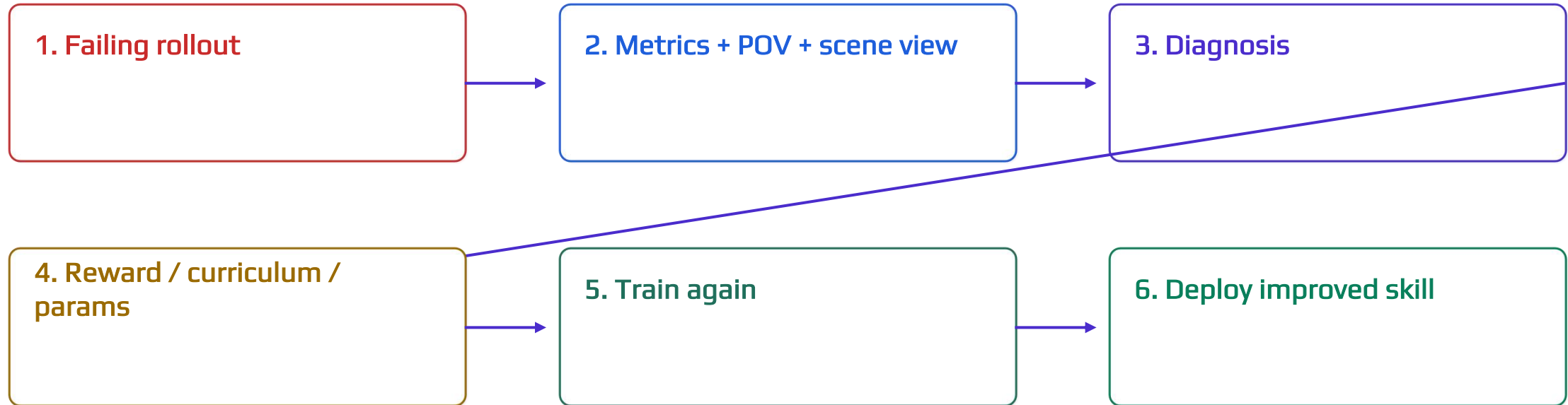


Code -> train -> deploy -> execute -> see -> diagnose -> remember -> evolve



The reusable pattern

No live demo today — but this is the loop to build and evaluate.



Do not only show final success. Show the failure, the diagnosis, the intervention, and the improved skill.



Thanks!

charbel.dandjinou@gmail.com
Charbel DANDJINO

